OSINARA · ТЕХНИЧЕСКАЯ ДОКУМЕНТАЦИЯ

Как устроена память агента

Полная схема подсистемы: что и куда кладётся при записи, как это ищется при чтении, как живут нити памяти и почему всё это разделено по областям доступа.

> Память — это не одна сущность, а три разных слоя с разными владельцами и разным сроком жизни. Eve отвечает только за верхний. Всё, что помнится между разговорами, живёт в PostgreSQL и принадлежит приложению.

ПУТЬ ОДНОГО СООБЩЕНИЯ



смысл оценивает только один агент и только один раз — фоновых LLM-пайплайнов нет

КОДОВАЯ БАЗА

ветка fix/memory-review-agent-collision · 1004d58

СТЕК

Eve 0.32.0 · PostgreSQL + pgvector · E5-small 384d

ДАТА

15 августа 2026

§ Содержание

11 РАЗДЕЛОВ

Документ идёт снизу вверх: сначала физика хранения, затем запись, индексация и чтение, потом нити и групповое ревью, в конце — границы доступа и карта таблиц.

1 **Три слоя памяти**
Активный контекст Eve, журнал переписки и долговременные claims — кто чем владеет

2 **Запись**
Контракт remember, проверка источника и одна атомарная транзакция

3 **Индексация**
Чанкинг, локальный E5 и векторы в pgvector

4 **Чтение**
Гибридный поиск в три ветки, RRF, профиль и сборка блока промпта

5 **Нити памяти**
Identity, роли, активация, дословные брифы и защита от дублей

6 **Групповое ревью**
Дорожки, пакеты по 50 сообщений и тихие фоновые ходы

7 **Изменение и удаление**
Reinforcement, правки с версиями, конфликты и undo

8 **Границы доступа**
Личное, семейное, групповое и единственное исключение

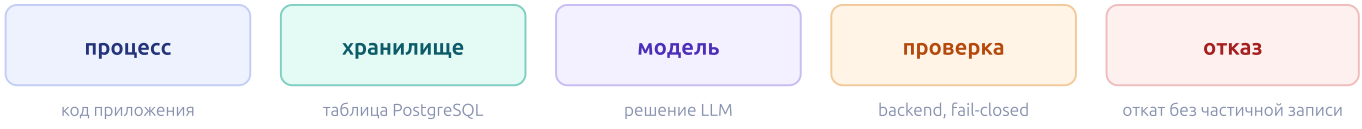
9 **Инструменты агента**
Тринадцать инструментов и кто из них кому доступен

10 **Карта данных**
Таблицы PostgreSQL и связи между ними

11 **Константы и коды ошибок**
Все пороги, бюджеты и квоты в одном месте

≡ **Главная идея**
Почему архитектура выглядит именно так — на последней странице

Как читать схемы



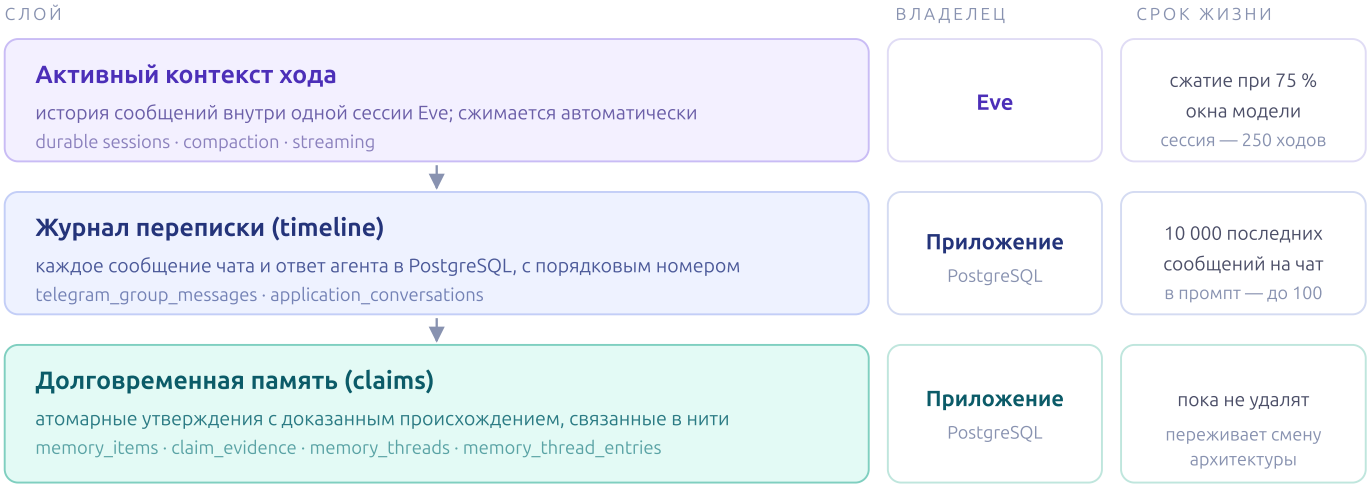
КЛЮЧ КО ВСЕМУ ДОКУМЕНТУ

Модель никогда не видит настоящих идентификаторов базы и не выбирает, в какую область памяти писать. Она оперирует непрозрачными ссылками `mem_*`, `thread_*`, `subj_*`, `entry_*`, а `familyId`, `userId`, `groupId`, роль и scope backend выводит из проверенного Telegram-контекста и актуального состояния PostgreSQL.

1 Три слоя памяти

ФИЗИКА ХРАНЕНИЯ

Слово «память» в проекте означает три разные вещи. Их легко перепутать, но у них разные владельцы, разные форматы и разный срок жизни. Eve отвечает только за верхний слой; два нижних целиком принадлежат приложению.



Слои памяти. Стрелки означают «переживает»: контекст хода умирает вместе с сессией, журнал живёт дольше, claims — дольше всех. Информация поднимается снизу вверх при каждом ходе и опускается сверху вниз только через явный вызов `remember`.

СЛОЙ	ГДЕ ЛЕЖИТ	КТО ВЛАДЕЕТ	СКОЛЬКО ЖИВЁТ
Активный контекст	сессия Eve, история сообщений модели	Eve	сжатие при 75 % окна; сессия — до 250 завершённых ходов, 30 дней бездействия, 90 дней хранения
Журнал переписки	`telegram_group_messages`, `application_conversations`	приложение	последние 10 000 сообщений на разговор; в контекст модели — до 100 сообщений и 12 000 символов
Долговременная память	`memory_items` + доказательства + нити	приложение	пока запись не удалят явно

ПОЧЕМУ ЭТО РАЗДЕЛЕНИЕ ВАЖНО

Журнал переписки — это **проверенный источник**: только на него можно ссылаться при сохранении факта. Активный контекст модели источником быть не может, потому что он уже прошёл через сжатие и в нём перемешаны данные и рассуждения. Поэтому источник для записи всегда резолвится по журналу, а не по тому, что модель «помнит» в текущем окне.

2 Запись: как факт попадает в память

WRITE FLOW

Решение о сохранении принимает сам основной чат-агент, в том же ходе, где пришло сообщение. Никакого фонового «экстрактора», который вторым проходом LLM перечитывает переписку, больше нет — он вырезан миграцией `059_main_agent_owned_memory.sql`.

Контракт инструмента `remember`

Что агент указывает обязательно

content	атомарный текст, до 4000 символов
kind	profile · preference · fact · episode · family_shared
scope	personal · family · group — только доступный в этом чате
sensitivity	normal · sensitive
basis	agent_inferred (сам решил) или user_requested (попросили явно)
subject	о ком запись: current_author, verified_ref, label или none

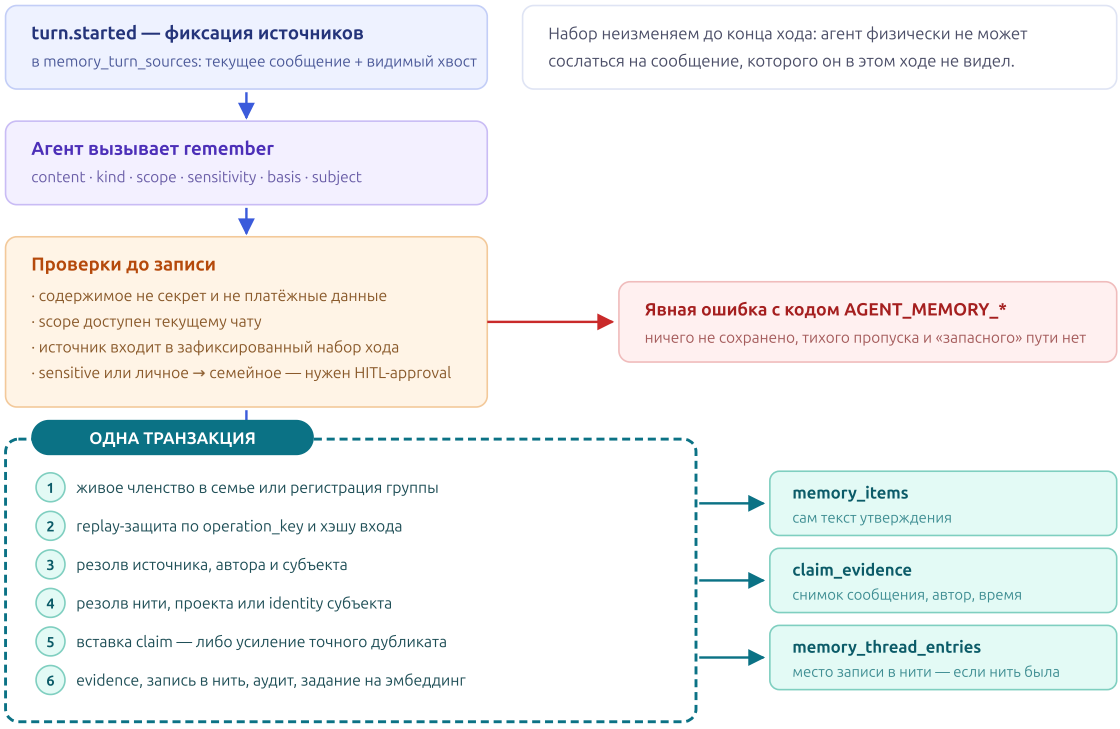
Что можно указать дополнительно

thread	создать нить или прикрепиться к существующей — тем же вызовом
sourceSequence	номер сообщения из видимого хвоста; только в группах

Чего указать нельзя вообще

ID пользователя, семьи, группы, разговора; ID сообщения-источника; Telegram message id; чужой scope. Всё это backend берёт сам — из проверенного канала и базы. Попытка передать такое поле не пройдёт схему.

Полный путь одной записи



> **Путь записи.** Всё, что внутри пунктирной рамки, происходит в одной транзакции PostgreSQL. Если падает любой шаг — включая действие с нитью — откатывается весь claim целиком. Состояния «факт сохранился, а в нить не попал» не существует.

Проверка источника

На старте каждого хода backend фиксирует в `memory_turn_sources` и `memory_turn_source_sets` неизменяемый набор: какое сообщение является текущим и какие сообщения хвоста видимы агенту (`agent/lib/memory-turn-source.ts`). Когда приходит `remember`, источник резолвится только внутри этого набора, и заново сверяется всё: тот ли разговор, тот ли автор, совпадает ли партиция scope с авторизацией. В личном чате ссылаться на прошлые или чужие сообщения нельзя в принципе.

В `claim_evidence` ложится снимок исходного сообщения (до 1000 символов), метка автора, время наблюдения и вид свидетельства:

ВИД	ЧТО ОЗНАЧАЕТ
`firsthand`	прямое заявление проверенного автора о себе
`reported`	другой участник рассказал о ком-то третьем — это не подтверждение субъекта
`inferred`	вывод модели из источника, а не прямое утверждение

ПОДТВЕРЖДЕНИЕ ВЛАДЕЛЬЦА (HITL)

Запись с `sensitivity: sensitive`, а также запись из личного чата в семейную область (раскрытие в более широкую зону) требуют явного подтверждения до вызова репозитория. Отказ терминален для текущего действия. Пароли, токены, API-ключи, приватные ключи, коды подтверждения и платёжные реквизиты режет content policy — независимо от любого подтверждения.

Квоты по областям

personal	5 000 записей
family	20 000 записей
group	10 000 записей

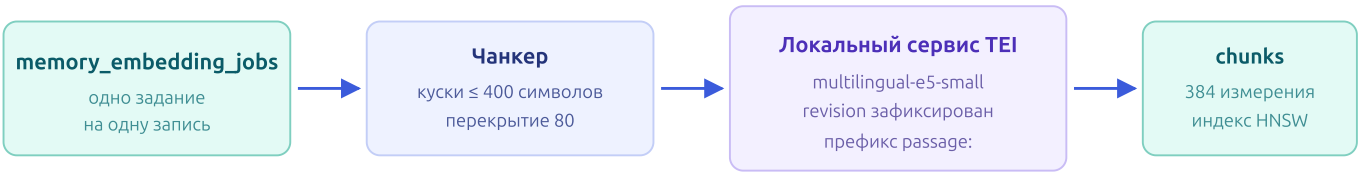
Защита от повторов

Пара `(family_id, operation_key)` плюс хэш канонического входа делают повторный вызов идемпотентным: тот же результат, без второй записи. Если при повторе изменился вход — `AGENT_MEMORY_REPLAY_MISMATCH`. Записи операций переживают удаление claim, чтобы повтор не воскресил удалённое.

3 Индексация: как память становится искомой

ЛОКАЛЬНЫЙ E5

Сохранённый текст сам по себе не ищется по смыслу. Чтобы это стало возможным, отдельный воркер превращает его в векторы — полностью локально, без единого обращения к внешнему провайдеру.



Правила воркера

· аренда задания на 120 секунд, партия из 4 записей, к провайдеру — партиями по 8 кусков
· запись «всё или ничего»: либо коммитятся все куски одной записи, либо ни одного
· ошибка фиксируется одним терминальным статусом; скрытых повторов нет

> **Конвейер индексации** `scripts/memory-embedding-worker.ts`. Правки и удаления записи сбрасывают то же самое задание обратно в `pending`, поэтому индекс не расходится с текстом.

ГРАНИЦЫ ОТВЕТСТВЕННОСТИ ЭМБЕДДИНГОВ

Векторы влияют **только на поиск и активацию нитей**. Они никогда не влияют на истинность записи и никогда — на права доступа. Похожесть двух claims не делает их дубликатами, не объединяет их и не отменяет один другим. Это отдельное осознанное решение архитектуры, а не недоделка.

Почему именно 400 символов

У E5 жёсткий лимит в 512 токенов. Ограничение по символам с запасом гарантирует, что в лимит уложится даже текст, набитый пунктуацией и эмодзи, где токенов на символ приходится намного больше обычного.

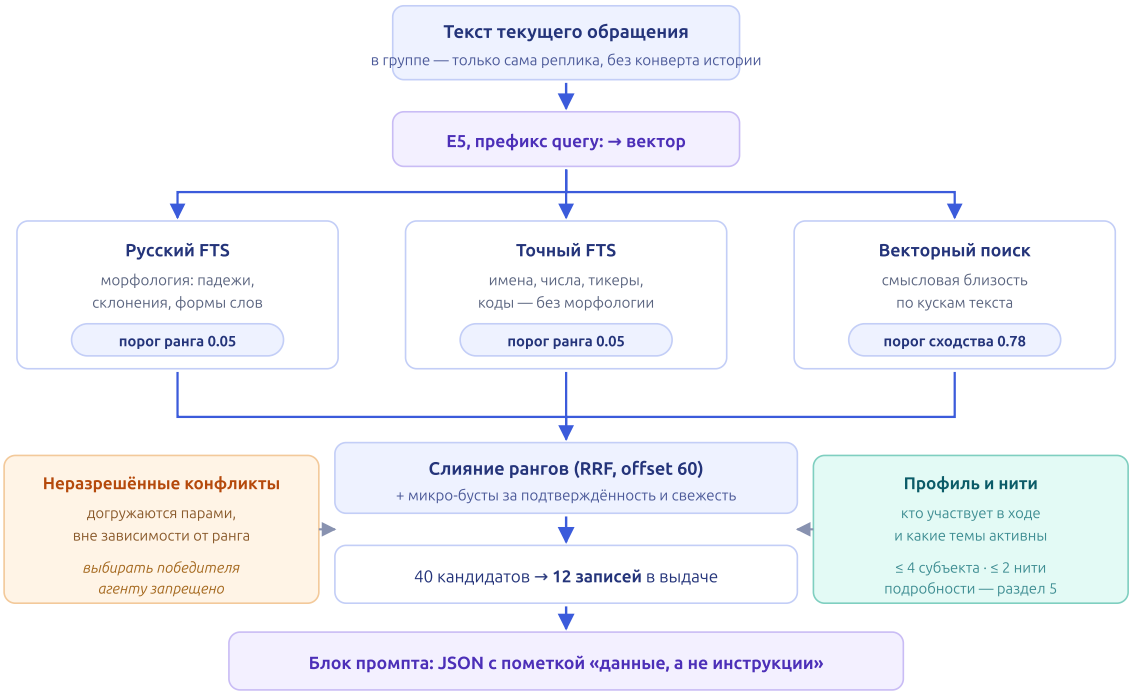
Почему модель прибита к revision

Версия модели хранится вместе с вектором. Смена модели означала бы другое векторное пространство, где старые и новые векторы несравнимы. Поэтому апгрейд — это не «подменить сервис», а осознанная переиндексация всей базы (`scripts/reindex-memory.ts`).

4 Чтение: что агент получает в начале хода

RETRIEVAL

Агент не ищет по памяти сам «по ключевым словам». К моменту, когда он видит сообщение, сервер уже выполнил поиск, отфильтровал результаты по правам доступа и вложил их в промпт отдельным блоком.



> **Конвейер поиска** `agent/lib/memory-retrieval-repository.ts`. Три ветки работают независимо и по-разному ошибаются: точный FTS вытащит «SBER» и номер счёта там, где морфология и смысл промахнутся, а вектор найдёт перефразированное там, где не совпало ни одно слово. Каждая ветка отсекается своим порогом до объединения, поэтому нерелевантный запрос честно возвращает пустоту.

Что ещё происходит при сборке блока

- **Схлопывание дубликатов только при чтении.** Одинаковые по смыслу повторы объединяются в выдаче, но в хранилище ничего не меняется.
- **Двойная проверка прав.** Один предикат авторизации встроен прямо в SQL (до ранжирования), второй срабатывает перед отдачей текста в промпт. Отозвали членство или удалили группу — устаревший scope в сессии Eve уже ничего не даст.
- **Профильный блок.** Проекция активных записей о людях, релевантных этому ходу: текущий автор, кого упомянули, кому ответили, кто всплыл в поиске. До 4 субъектов, до 30 записей на субъекта, до 12 000 символов. Именно отсюда агент берёт `subj_*`, когда сохраняет факт о другом человеке.
- **Углубление по требованию.** Если выдачи мало для сложного вопроса, агент может сделать до трёх последовательных вызовов `search_memories` — каждый по ещё не выясненному аспекту, а не перефразировкой предыдущего.

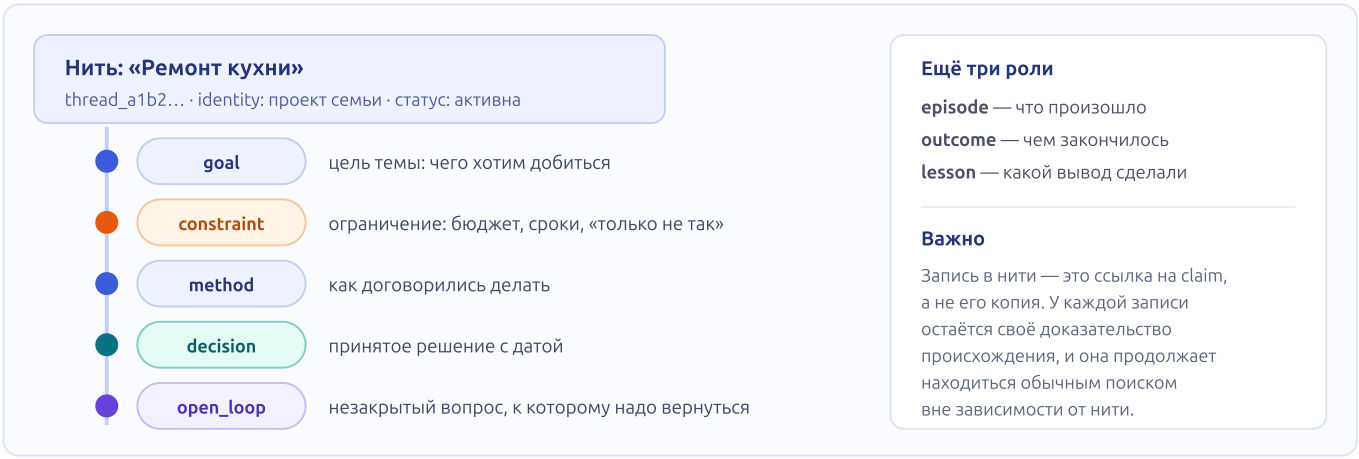
ОТДЕЛЬНО ПРО ОТКАЗ

Если сборка блока памяти упала, в промпт кладётся честное `AGENT_MEMORY_UNAVAILABLE: память недоступна` — а не пустой список. Разница принципиальная: пустой список агент прочитал бы как «записей нет» и уверенно сказал бы пользователю неправду. Формулировка блока прямо запрещает утверждать, что память проверена.

5 Нити памяти

THREADS

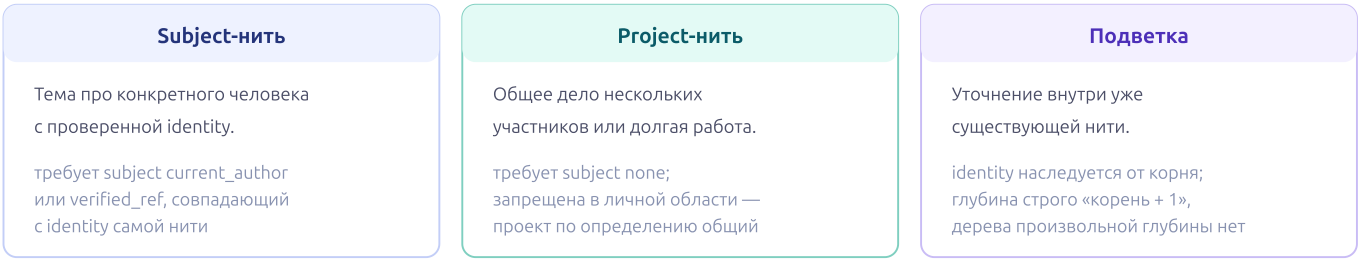
Отдельные факты хорошо отвечают на вопрос «что мы знаем», но плохо — на вопрос «как мы к этому пришли». Нить склеивает разрозненные записи в одну длинную тему с целью, ограничениями, решениями и открытыми вопросами.



> **Анатомия нити.** Восемь ролей задают, чем именно запись является для темы. Роль указывает агент в момент вызова `remember`; она же определяет приоритет показа при активации.

Identity: чьей темой является нить

У каждой нити ровно один «владелец смысла» внутри одной партиции доступа. Это не украшение, а то, что не даёт темам разных людей и групп смешаться.



ТОНКОСТЬ, КОТОРУЮ ЛЕГКО ПРОПУСТИТЬ

Прикрепление записи к чужой нити **не назначает субъекта записи молча**. Если агент явно указал субъекта, а identity нити другая — операция отклоняется, а не «подстраивается». Свободная текстовая метка (`subject: label`) вообще не может задавать identity нити: она сохраняет только подпись внутри claim и не попадает в проверенный профиль.

Защита от разрастания: два разных порога

Главный риск нитей — сотня почти одинаковых тем «Ремонт», «Ремонт кухни», «Кухня ремонт». Поэтому перед созданием новой нити backend под блокировкой ищет до трёх похожих активных кандидатов.



Почему пороги разные

У коротких названий высокий «фоновый» уровень схожести: два несвязанных заголовка из трёх слов часто дают 0.7–0.8.

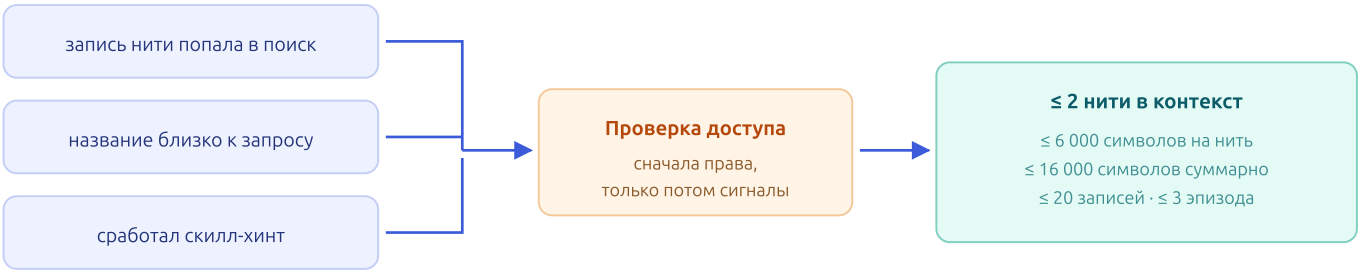
Порог поиска можно держать мягким — лишний кандидат не страшен. Порог блокировки обязан быть жёстким: ложное срабатывание останавливае

Если сработал порог по названию (≥ 0.92) или по описанию цели (триграммная близость ≥ 0.9), создание отклоняется с кодом `AGENT_MEMORY_THREAD_CANDIDATE_EXISTS`, весь claim откатывается, а агенту возвращаются непрозрачные ссылки кандидатов. Дальше сценарий строго ограничен:

1. Агент сверяет названия кандидатов с текущей темой, при необходимости читает подозрительного кандидата через `read_memory_thread`.
2. Совпало — повторяет `remember` с `action: attach` к нужной ссылке.
3. Не совпало — ровно **одна** уточнённая попытка создать заведомо другую тему.
4. Если и она вернула кандидата — `AGENT_MEMORY_THREAD_RETRY_EXHAUSTED`. Разрешён один терминальный attach, дальше — честное «отдельную нить создать не удалось».

Попытки живут в `memory_thread_creation_attempts` с пятиминутной арендой, поэтому два параллельных вызова не создадут две одинаковые нити: второй сериализуется на той же блокировке источника.

Активация: как нить попадает в контекст



Порядок показа записей внутри нити фиксирован: ограничения и конфликты → цели и открытые вопросы → методы → решения и результаты → уроки → эпизоды. Если бюджет кончился, следующая запись пропускается целиком, а не обрезается.

> **Активация нити.** Скилл-хинт — это связь вида «вызвали скилл `t-invest` → подтянуть нить «Инвестиции»». Проверка доступа стоит раньше сигналов специально: спроецированная наружу запись не должна активировать нить из области, к которой у этого чата доступа нет.

БРИФЫ НИТЕЙ НЕ ПЕРЕСКАЗЫВАЮТСЯ

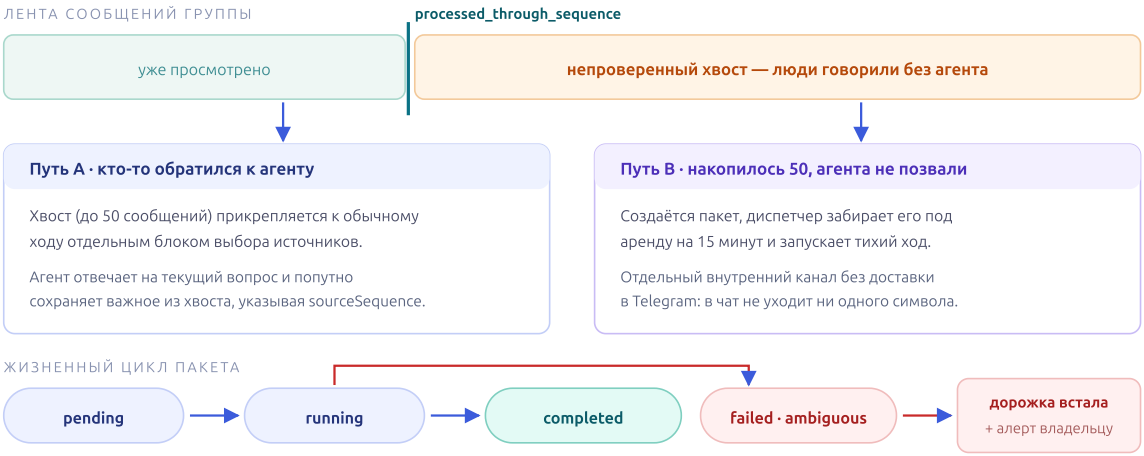
Это ключевое отличие от типовых реализаций «долгой памяти». Никакая LLM не сжимает нить в резюме. Репозиторий берёт исходные записи с их доказательствами и вставляет их **дословно**. Причина простая: пересказ — это место, где в память бесшумно попадает выдуманный факт. Модуль-генератор называется `memory-thread-brief-generator.ts` по историческим причинам, никакой генерации внутри нет.

6 Групповое ревью памяти

СВЕЖАЯ ПОДСИСТЕМА

Проблема: в семейной или внешней группе люди переписываются между собой и агента не зовут. Всё это проходит мимо памяти — а там ровно та жизнь, ради которой агент и существует.

Решение — «дорожка» (`memory_review_lanes`) на каждый разговор и топик. Дорожка помнит одно число: до какого порядкового номера сообщения память уже просмотрена. Дальше есть два пути, как хвост непроверенных сообщений всё-таки попадёт агенту.



> **Дорожки и пакеты.** Пакеты одной дорожки образуют непрерывную цепочку: каждый следующий начинается ровно там, где закончился предыдущий. Пропустить кусок переписки «между пакетами» невозможно по построению.

Что видит агент в тихом ходе

Промпт фонового ревью намеренно узкий (`agent/lib/memory-review/memory-review-prompt.ts`):

- проверить ровно перечисленные сообщения — не более 50;
- ничего не отправлять в Telegram и ни к кому не обращаться;
- сохранять только с `basis: agent_inferred` и `sensitivity: normal`;
- чувствительное, секреты и платёжные данные — не сохранять вообще;
- нить создавать только при сильном сигнале: длительная цель, ожидаемые обновления, незакрытый вопрос, многошаговый проект;
- если сохранять нечего — завершить ход без единого вызова инструментов.

НЕОДНОЗНАЧНЫЙ СБОЙ ОСТАНАВЛИВАЕТ ДОРОЖКУ

Если непонятно, успел ли пакет отработать (упал ровно на передаче хода или на отметке о старте), дорожка **блокируется**, автоматический повтор запрещён, а владельцу уходит алерт. Логика та же, что и в durable-ingress Telegram: лучше остановиться и сказать человеку, чем вслепую повторить и задвоить записи или заплатить за модель дважды. Безопасных попыток восстановления — не больше одной.

- **Авторство.** При сохранении из хвоста `subject: current_author` означает автора **выбранного сообщения**, а не того, кто позвал агента. Иначе чужие факты приписывались бы тому, кто просто задал вопрос.
- **Чувствительное из хвоста не поднимается.** Такое сведение можно записать только из текущего сообщения самого автора и с подтверждением.

7 Изменение, конфликты и удаление

Автоматического «слияния по смыслу» в системе нет вообще. Классификатор семантических отношений удалён осознанно: похожесть двух формулировок — слишком слабое основание, чтобы молча объявить одну устаревшей.

ОПЕРАЦИЯ	КТО ИНИЦИИРУЕТ	ЧТО ПРОИСХОДИТ
Усиление (reinforcement)	сервер, автоматически	Точное совпадение нормализованного текста в той же проверенной identity — новая запись не создаётся, старая получает дополнительное доказательство
Правка	пользователь, явно	`manage_memory edit` по конкретному `mem_*`, с цепочкой версий
Удаление	пользователь, явно	физическое удаление записи; запись операции остаётся, чтобы повтор не воскресил удалённое
Отмена	пользователь, сразу	`undo` — только для немедленной отмены только что предложенного сохранения
Разрешение конфликта	пользователь + HITL	выбрать одну версию, оставить обе или зафиксировать «не решено» — по конкретной ссылке конфликта



Правки и удаления синхронно чинят связанное состояние: сбрасывают задание на переиндексацию, обновляют записи в нитях и проекции профилей — через триггеры базы, а не «когда-нибудь потом» фоновым процессом.

8 Границы доступа

SCOPES

Это не настройка приватности поверх памяти, а её несущая конструкция. Область выводится из типа чата и проверенного членства, а не из того, что попросила модель.

	ЛИЧНОЕ	СЕМЕЙНОЕ	ГРУППОВОЕ	
Личный чат подтверждённый участник семьи	**доступно**	**доступно**	нет	операции владельца только здесь
Семейная группа активный участник той же семьи	нет	**доступно**	нет	личное сюда не протекает
Внешняя группа изолированная зона доверия	**никогда**	**никогда**	**только своя** группа	смена типа группы стирает старую зону

Единственное исключение: владелец может явно, из личного чата, включить проекцию собственного профиля в конкретную внешнюю группу.

КАК ЭТО ОБЕСПЕЧИВАЕТСЯ ТЕХНИЧЕСКИ

- Область записи проверяется функцией `requireWritableScope` — модель не может попросить недоступную.
- Каждый SQL-запрос чтения несёт предикат живого доступа: членство, регистрация группы, роль — проверяются заново, а не берутся из сессии.
- Изменение типа группы пересоздаёт зону доверия и удаляет данные прежней области — старые записи не «перетекают» в новый режим.
- Операции владельца разрешены только в его личном чате; после HITL исполнитель заново перепроверяет роль в базе.

9 Инструменты агента

TOOL SURFACE

Инструменты выдаются не списком «на все случаи», а под конкретный режим чата: недоступный инструмент не имеет описания вообще, а не заменяется заглушкой. Модель не видит того, чем не может воспользоваться.

ИНСТРУМЕНТ	ТИП	ЧТО ДЕЛАЕТ
`remember`	запись	Один claim с доказательством и опциональным действием с нитью — атомарно. Доступен **только корневому агенту**: субагент не получает описания вообще
`search_memories`	чтение	Явный гибридный поиск, когда автоматической выдачи не хватило; до трёх последовательных запросов по разным аспектам
`list_memories`	чтение	Постраничный список записей, доступных этому чату
`get_memory_source`	чтение	Откуда взялась конкретная запись: автор, время, вид доказательства
`manage_memory`	мутация	Правка, удаление, немедленная отмена — по явной ссылке `mem_*`
`manage_memory_conflict`	мутация + HITL	Разрешение конфликта версий по ссылке конфликта
`search_memory_threads`	нити	Поиск нити по названию и цели, когда нужно прикрепиться к существующей теме
`list_memory_threads`	нити	Постраничный список нитей с непрозрачными ссылками
`read_memory_thread`	нити	Углубление в одну нить: до 20 записей и 12 000 символов на страницу
`manage_memory_thread`	нити + HITL	Завершить или переоткрыть нить, с защитой от повторного применения
`read_profile_view`	профиль	Чтение точного упорядоченного снимка профиля, если доступ не изменился
`manage_profile_projection`	владелец	Явное согласие на показ собственного профиля во внешней группе; только из личного чата
`export_memory`	экспорт	Выгрузка личной памяти в JSON и Markdown с доставкой файлами в Telegram; форматирование вне модели

Почему у субагента нет `remember`

Субагент запускается для сложной работы, которой нужен свежий контекст, и наследует проверенную авторизацию родителя. Но он видит только свой кусок задачи и не отвечает за разговор целиком. Право решать, что попадёт в долговременную память, оставлено там, где есть полная картина, — у корневого агента.

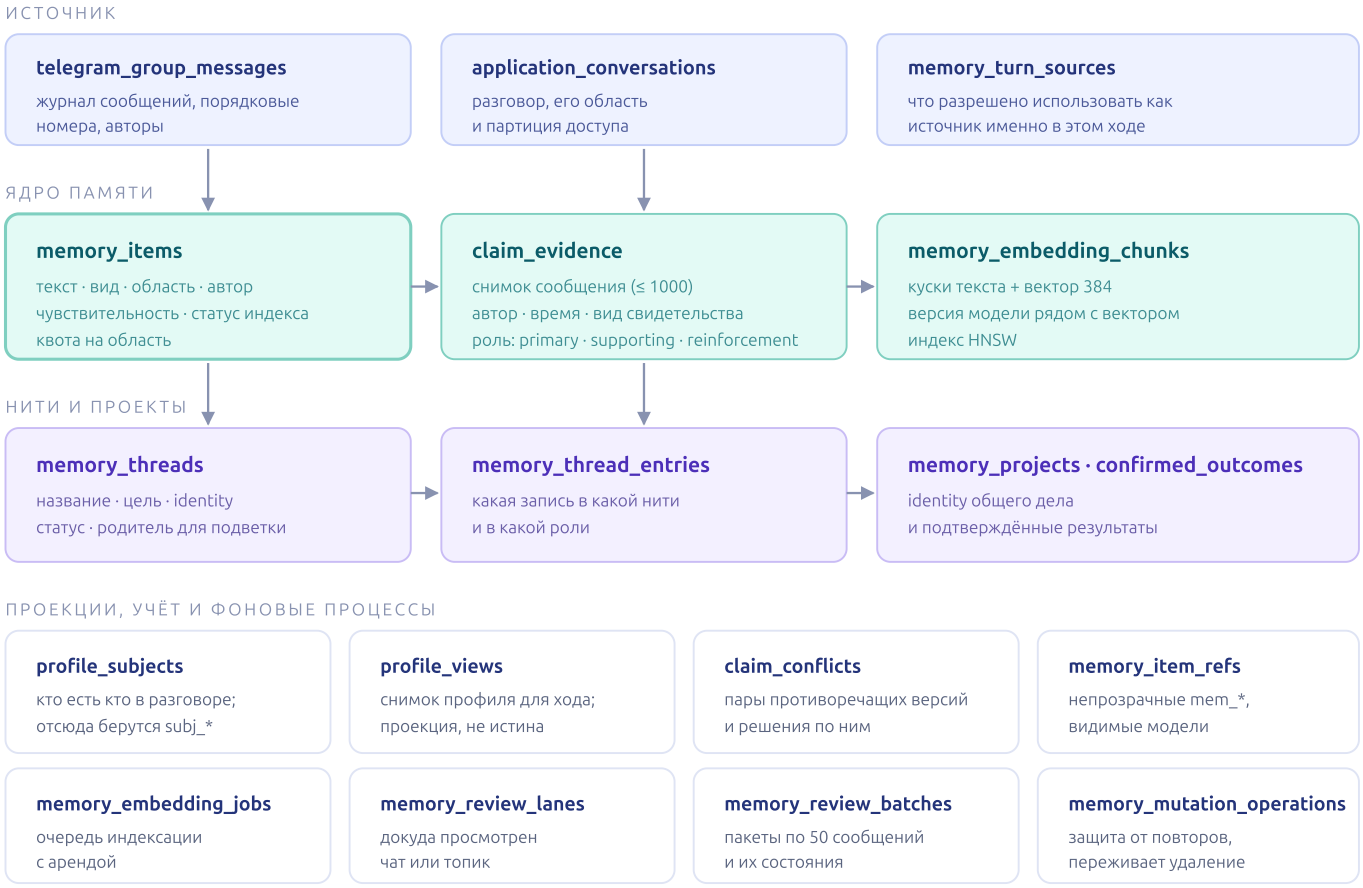
Как выдаётся набор

Через динамический резолвер, пересобираемый на каждый ход (`agent/tools/capabilities.ts`). Если режим чата не удалось доказать или резолвер упал — прикладных инструментов нет вообще. Это тот же принцип fail-closed, что и в блоке памяти.

10 Карта данных

POSTGRESQL

Основные таблицы подсистемы и связи между ними. Вспомогательные таблицы отставленных подсистем в схеме ещё есть, но рантайм в них не пишет.



> **Карта таблиц.** Верхний ряд — проверенный источник, средние — сама память, нижние — проекции и служебный учёт. Всё, что видит модель, — это ссылки из `memory_item_refs` и аналогичных таблиц; настоящие ключи не покидают backend.

11 Константы, бюджеты и коды ошибок

СПРАВОЧНИК

Все числа собраны в `agent/lib/memory-config.ts` и `agent/lib/memory-review/memory-review-config.ts`. В `.env` ничего из этого нет и быть не должно — это продуктовые настройки, а не окружение.

Поиск

кандидатов	40
в выдаче	12
порог лексики	0.05 для обеих FTS-веток
порог смысла	0.78
RRF offset	60

Нити

нитей в контекст	2
символов всего	16 000
на одну нить	6 000 · до 20 записей
эпизоды	3 шт., до 2 000 символов
блокировка дубля	0.92 по названию · 0.9 по цели
попыток создания	2, затем терминальный attach

Профиль

субъектов	4
записей на субъекта	30
символов всего	12 000
«спящий» субъект	60 дней без активности

Ревью и индексация

размер пакета	50 сообщений
аренда пакета	15 минут
попыток восстановления	1
аренда индексации	120 секунд
кусок текста	≤ 400 символов, перекрытие 80

Коды ошибок записи

КОД	КОГДА ВОЗНИКАЕТ
`AGENT_MEMORY_EXPLICIT_SOURCE_INVALID`	источник не входит в зафиксированный набор хода или не принадлежит этому автору
`AGENT_MEMORY_SCOPE_DENIED`	запрошенная область недоступна в текущем чате
`AGENT_MEMORY_THREAD_CANDIDATE_EXISTS`	нашлась слишком похожая существующая нить; claim откачен
`AGENT_MEMORY_THREAD_RETRY_EXHAUSTED`	исчерпаны две попытки создать нить по одному источнику
`AGENT_MEMORY_THREAD_NOT_FOUND`	ссылка на нить не существует или недоступна
`AGENT_MEMORY_REPLAY_MISMATCH`	повтор той же операции пришёл с изменённым содержимым
`AGENT_MEMORY_QUOTA_EXCEEDED`	исчерпана квота записей для области
`AGENT_MEMORY_UNAVAILABLE`	блок памяти собрать не удалось; агент обязан честно сказать об этом

Правило для всех кодов: стабильный код, понятное русское сообщение человеку, технические подробности — только в логах. Тихий пропуск, «запасное» сохранение и скрытый повтор невозможны.

Главная идея

Смысл оценивает один агент и ровно один раз

Тот же чат-агент, который читает сообщение и отвечает пользователю, решает и что запомнить. Нет второго прохода LLM по расшифровке, нет фонового извлекателя фактов, нет генератора пересказов. Меньше мест, где может незаметно появиться выдуманный факт — и меньше денег за токены, потраченных на перечитывание одного и того же.

Backend отвечает за доказуемость

- источник — только проверенное сообщение
- область и личность — только из базы
- запись — атомарна или её нет вовсе
- повтор безопасен, тихий сбой невозможен

Модель отвечает за смысл

- стоит ли это помнить вообще
- это новая тема или продолжение старой
- какая роль у записи внутри темы
- чего в памяти не хватает для ответа

Из этого разделения следует всё остальное: дословные брифы вместо пересказов, два разных порога похожести вместо одного, честное «память недоступна» вместо пустого списка, остановка дорожки вместо слепого повтора. Каждое из этих решений — отказ от удобного поведения в пользу того, которое нельзя тихо испортить.

Где смотреть дальше

ФАЙЛ	ЧТО ТАМ
`docs/memory-system-full.md`	живая спецификация; актуальна, но написана до подсистемы группового ревью
`agent/lib/memory-claim-writer.ts`	та самая единственная транзакция записи
`agent/lib/memory-retrieval-repository.ts`	гибридный поиск и слияние рангов
`agent/lib/memory-thread-write.ts`	identity нитей, кандидаты, подветки
`agent/lib/memory-thread-brief-repository.ts`	активация нитей и сборка дословного контекста
`agent/lib/memory-review/`	дорожки, пакеты, диспетчер и восстановление
`agent/lib/prompt/turn-blocks.ts`	сборка блоков промпта, включая fail-closed поведение